Tribhuvan University

Faculty of Humanities & Social Sciences

OFFICE OF THE DEAN

Software Engineering

BCA — Semester 4 — 2019 — Answer Sheet

Subject Code: CACS253

Group B

Attempt any SIX questions. [6×5=30]

2. What are the attributes of good software? What are the key challenges that software engineering faces during software development? Explain.

[5]

Answer: Attributes of Good Software:

1. Maintainability: Software should be written so that it can evolve to meet changing customer needs. Clean code, documentation, and modular design help.

2. Dependability: Software must be reliable, secure, and safe. It should not cause physical or economic damage in case of failure.

3. Efficiency: Software should not make wasteful use of system resources such as memory and processor cycles.

4. Usability: Software must be easy to use for the intended users. Good UI/UX design, appropriate documentation, and intuitive workflows are essential.

5. Portability: Software should be able to run on different platforms and environments with minimal changes.

Key Challenges in Software Engineering:

1. Changing Requirements: Requirements often change during development, leading to rework and schedule slips.

2. Increasing Complexity: Modern systems are large, distributed, and integrated with legacy systems.

3. Time and Cost Constraints: Pressure to deliver quickly while maintaining quality.

4. Quality Assurance: Ensuring the software is bug-free, secure, and performant across diverse environments.

5. Legacy System Integration: Maintaining and integrating with older systems while adopting new technologies.

6. Security Threats: Protecting against cyber attacks, data breaches, and ensuring privacy compliance.

7. Team Coordination: Managing distributed teams, communication gaps, and knowledge silos.

8. Technology Evolution: Keeping up with rapidly changing frameworks, languages, and platforms.

3. What is software process model? List the types of software model. Explain agile methods and their principles.

[5]

Answer: Software Process Model:
A software process model is a simplified representation of a software process. It defines the workflow, activities, and phases involved in software development from conception to retirement. It provides structure and discipline to the development effort.

Types of Software Process Models:
1. **Waterfall Model** — Linear sequential approach.
2. **Incremental Model** — Build the system in increments.
3. **Iterative Model** — Repeat cycles of design, build, test.
4. **Spiral Model** — Risk-driven iterative model.
5. **V-Model** — Verification and validation parallel phases.
6. **Agile Model** — Flexible, iterative, customer-centric.
7. **Prototyping Model** — Build prototypes to clarify requirements.
8. **Rad Model (Rapid Application Development)** — Fast development with user involvement.

Agile Methods and Principles:
Agile is an iterative approach to software delivery that builds solutions incrementally from the start of the project, instead of trying to deliver it all at once near the end.

Key Agile Methods:
• **Scrum:** Uses sprints (2-4 weeks), daily stand-ups, product backlog, and Scrum roles (Product Owner, Scrum Master, Team).
• **Extreme Programming (XP):** Emphasizes pair programming, continuous integration, test-driven development, and frequent releases.
• **Kanban:** Visual workflow management using boards and WIP limits.

Twelve Principles of Agile Manifesto:
1. Highest priority is to satisfy the customer through early and continuous delivery.
2. Welcome changing requirements, even late in development.
3. Deliver working software frequently (weeks rather than months).
4. Business people and developers must work together daily.
5. Build projects around motivated individuals; give them the environment and support they need.
6. Face-to-face conversation is the most efficient method of conveying information.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity — the art of maximizing the amount of work not done — is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective and adjusts accordingly.

4. What are requirements? Explain functional, non-functional, domain and organizational requirements.

[5]

Answer: Requirements:
Requirements are descriptions of what a software system should do and the constraints under which it must operate. They form the basis for software design, implementation, testing, and acceptance.

Types of Requirements:

1. Functional Requirements:
These describe the specific services the system should provide, how it should react to particular inputs, and what it should not do.
• Example: "The system shall allow users to search for products by name, category, or price range."
• Example: "The system shall generate an invoice after each purchase."

2. Non-Functional Requirements:
These define system qualities, constraints, and performance criteria rather than specific behaviors.
• **Performance:** "The system shall process 1000 transactions per second."
• **Security:** "All user passwords must be encrypted using AES-256."
• **Reliability:** "System uptime must be 99.9%."
• **Usability:** "New users shall complete onboarding in under 5 minutes."

3. Domain Requirements:
These are requirements that come from the application domain of the system. They reflect characteristics of that domain and may be functional or non-functional.
• Example (Healthcare): "The system must comply with HIPAA regulations for patient data."
• Example (Banking): "All transactions must support double-entry bookkeeping."

4. Organizational Requirements:
These are broad system requirements derived from policies and procedures in the customer's and developer's organizations.
• **Operational:** "The system must run on the organization's existing Linux servers."
• **Development:** "All code must follow the company's Java coding standards."
• **Environmental:** "The system must use the organization's single sign-on (SSO) provider."
• **Regulatory:** "The software must comply with GDPR for EU users."

Summary: Functional requirements define what the system does; non-functional define how well; domain requirements reflect industry-specific rules; organizational requirements reflect company policies.

5. What is modularization? Differentiate cohesion and coupling.

[5]

Answer: Modularization:
Modularization is the process of dividing a software system into separate, independent modules (components or subsystems). Each module is a self-contained unit that performs a specific function and interacts with other modules through well-defined interfaces. The goals are to reduce complexity, improve maintainability, enable parallel development, and promote code reuse.

Difference between Cohesion and Coupling:
AspectCohesionCoupling DefinitionMeasures how closely related and focused the responsibilities of a single module are.Measures the degree of interdependence between different modules. GoalHigh cohesion is desired (each module does one thing well).Low coupling is desired (modules are as independent as possible). TypesCoincidental, Logical, Temporal, Procedural, Communicational, Sequential, Functional.Content, Common, External, Control, Stamp, Data. Effect on maintenanceHigh cohesion makes a module easier to understand, modify, and test.Low coupling reduces ripple effects when one module is changed. ExampleA module that calculates tax (functional cohesion) is better than one that mixes tax, reporting, and email (coincidental).Two modules sharing only necessary data (data coupling) is better than one controlling another's logic (control coupling).
Conclusion: Good software design aims for **high cohesion and low coupling** to create maintainable, scalable systems.

6. Why is UI design important? How is UI design visualized? Discuss.

[5]

Answer: Importance of UI Design:

1. First Impression: The UI is the first thing users see. A well-designed UI creates trust and encourages users to engage with the system.

2. Usability: Good UI design makes software easy to learn and use, reducing training time and support costs.

3. Productivity: Intuitive layouts and workflows help users complete tasks faster and with fewer errors.

4. Accessibility: Proper UI design ensures the software is usable by people with disabilities (screen readers, keyboard navigation, color contrast).

5. Competitive Advantage: A polished UI differentiates the product in the market and increases customer satisfaction.

6. Reduced Errors: Clear labels, feedback, and validation prevent user mistakes and data entry errors.

How UI Design is Visualized:

1. Wireframes: Low-fidelity sketches that show layout, content placement, and navigation without color or graphics. Tools: Balsamiq, Sketch.

2. Mockups: High-fidelity static designs that include colors, typography, and imagery. They show the final look and feel.

3. Prototypes: Interactive simulations of the UI that allow stakeholders to click through screens and experience the flow. Tools: Figma, Adobe XD.

4. Storyboards: Visual narratives showing how users interact with the system in real-world scenarios.

5. User Flow Diagrams: Flowcharts that map the steps a user takes to complete a task, identifying decision points and screens.

6. Design Systems: Collections of reusable components (buttons, forms, menus) with defined standards for consistency across the application.

UI Design Principles:

- **Consistency:** Uniform design language throughout the application.
- **Feedback:** Inform users about actions (loaders, success messages).
- **Simplicity:** Reduce clutter; focus on essential elements.
- **Flexibility:** Accommodate both novice and expert users (shortcuts).
- **Recovery:** Allow undo and error correction.

7. Why is software testing considered as major component in SDLC? Explain software testing.

[5]

Answer: Why Software Testing is a Major Component in SDLC:

1. Quality Assurance: Testing ensures the software meets specified requirements and performs as expected.
2. Bug Detection: Identifies defects early, reducing the cost of fixing them later in the lifecycle.
3. Risk Mitigation: Prevents failures in production that could cause financial loss, data breaches, or safety hazards.
4. Customer Satisfaction: Delivering reliable software builds trust and meets user expectations.
5. Validation and Verification: Confirms that the product is built correctly (verification) and is the right product (validation).
6. Compliance: Ensures adherence to regulatory and industry standards.

Explanation of Software Testing:

Software testing is the process of evaluating a software application to find differences between given input and expected output. It includes manual and automated activities to verify and validate software.

Levels of Testing:

1. Unit Testing: Tests individual components or functions in isolation.
2. Integration Testing: Tests combined parts of the application to ensure they work together.
3. System Testing: Tests the complete integrated system against requirements.
4. Acceptance Testing: Validates the software against business needs; includes UAT (User Acceptance Testing).

Types of Testing:

• **Functional Testing:** Black-box testing of features (unit, integration, system).
• **Non-Functional Testing:** Performance, security, usability, compatibility.
• **Regression Testing:** Ensures new changes do not break existing functionality.
• **Smoke Testing:** Basic checks to verify the build is stable enough for further testing.

Testing Process:

1. Test Planning → 2. Test Design → 3. Test Execution → 4. Defect Reporting → 5. Test Closure.

Conclusion: Testing is integral to SDLC because it directly impacts software quality, reliability, and user satisfaction.

8. What is project management? Why is it important? Explain.

[5]

Answer: Project Management:
Project management is the application of knowledge, skills, tools, and techniques to project activities to meet project requirements. In software engineering, it involves planning, organizing, staffing, directing, and controlling the development of software systems within constraints of time, cost, scope, and quality.

Why Project Management is Important:
1. On-Time Delivery: Ensures projects are completed within deadlines through scheduling (Gantt charts, PERT, CPM).
2. Budget Control: Manages costs and resources to prevent overruns.
3. Scope Management: Defines and controls what is included in the project, preventing scope creep.
4. Quality Assurance: Integrates quality standards and testing into the development process.
5. Risk Management: Identifies potential risks early and develops mitigation strategies.
6. Resource Optimization: Allocates human, technical, and financial resources efficiently.
7. Stakeholder Communication: Maintains transparency and alignment between clients, developers, and management.
8. Team Coordination: Facilitates collaboration, resolves conflicts, and keeps the team motivated.

Key Knowledge Areas (PMBOK):
• Integration Management
• Scope Management
• Time Management
• Cost Management
• Quality Management
• Human Resource Management
• Communications Management
• Risk Management
• Procurement Management
• Stakeholder Management

Conclusion: Without effective project management, software projects are likely to fail due to missed deadlines, budget overruns, poor quality, and unmet user needs.

Group C

Attempt any TWO questions. [2×10=20]

9. Explain different software projects and their characteristics.

[10]

Answer: Types of Software Projects and Their Characteristics:

1. Web Development Projects:
• Delivered via browsers; responsive design required.
• Technologies: HTML, CSS, JavaScript, React, Angular, Node.js.
• Characteristics: Rapid deployment, continuous updates, SEO considerations, cross-browser compatibility.
• Examples: E-commerce sites, blogs, SaaS platforms.

2. Mobile Application Projects:
• Target iOS and/or Android platforms.
• Technologies: Swift, Kotlin, Flutter, React Native.
• Characteristics: Touch-based UI, offline capability, push notifications, app store compliance, device fragmentation.
• Examples: Fitness trackers, food delivery apps, mobile banking.

3. Embedded Systems Projects:
• Software runs on dedicated hardware with limited resources.
• Technologies: C, C++, Assembly, RTOS.
• Characteristics: Real-time constraints, memory optimization, hardware interfacing, safety-critical requirements.
• Examples: Automotive ECU, medical devices, IoT sensors.

4. Enterprise Software Projects:
• Large-scale systems for business operations.
• Technologies: Java, .NET, Oracle, SAP.
• Characteristics: High security, scalability, integration with legacy systems, multi-tenancy, regulatory compliance.
• Examples: ERP, CRM, HR management systems.

5. Data Science / AI Projects:
• Focus on data processing, machine learning, and predictive analytics.
• Technologies: Python, R, TensorFlow, PyTorch, SQL.
• Characteristics: Large data volumes, experimentation-driven, model training and validation, deployment challenges (MLOps).
• Examples: Recommendation engines, fraud detection, chatbots.

6. Game Development Projects:
• Highly interactive, graphics-intensive applications.
• Technologies: Unity, Unreal Engine, C#, C++.
• Characteristics: Real-time rendering, physics simulation, multiplayer networking, asset management.
• Examples: Mobile games, AAA console games, VR experiences.

7. Open Source Projects:
• Developed collaboratively by a community.
• Characteristics: Transparent development, public codebase, community governance, diverse contributor base.
• Examples: Linux, Apache, Mozilla Firefox.

Comparison Table:

TypeTeam SizeTimelineRisk LevelKey Challenge WebSmall-MediumShort-MediumMediumBrowser compatibility MobileSmall-MediumShortMediumDevice fragmentation EmbeddedSmallMedium-LongHighResource constraints EnterpriseLargeLongHighIntegration complexity Data/AIMediumMediumMediumData quality & model accuracy GameMedium-LargeLongHighPerformance optimization

10. Explain Black box testing and White box testing.

[10]

Answer: Black Box Testing:
Black box testing treats the software as a "black box" where the internal structure/design/implementation is not known to the tester. Tests are based on requirements and specifications.

Characteristics:
• Focuses on inputs and expected outputs.
• No knowledge of internal code required.
• Tests functional and non-functional requirements.
• Performed by QA teams and end-users.

Techniques:
• **Equivalence Partitioning:** Divide input data into valid and invalid partitions.
• **Boundary Value Analysis:** Test values at boundaries (min, max, just inside/outside).
• **Decision Table Testing:** Map business rules to test cases.
• **State Transition Testing:** Test different states of the application.
• **Use Case Testing:** Validate end-to-end user scenarios.

Advantages: Unbiased testing, simulates real user behavior, no programming knowledge needed.
Disadvantages: Cannot test all paths, difficult to identify root cause of failures.

White Box Testing:
White box testing examines the internal structure, logic, and code of the software. Testers need programming knowledge and access to source code.

Characteristics:
• Focuses on code coverage (statements, branches, paths).
• Validates internal security, data flow, and control flow.
• Performed by developers.

Techniques:
• **Statement Coverage:** Every statement is executed at least once.
• **Branch Coverage:** Every decision branch (true/false) is tested.
• **Path Coverage:** All possible paths through the code are tested.
• **Condition Coverage:** Each boolean sub-expression is evaluated to true and false.
• **Loop Testing:** Test loops for zero, one, many, and maximum iterations.

Advantages: Thorough testing of internal logic, early bug detection, optimization opportunities.
Disadvantages: Time-consuming, expensive, requires skilled testers, may miss missing features.

Comparison:
AspectBlack BoxWhite Box Knowledge of codeNot requiredRequired FocusExternal behaviorInternal logic TestersQA / End usersDevelopers Coverage metricRequirements coverageCode coverage When performedSystem / Acceptance testingUnit / Integration testing
Conclusion: Both techniques are complementary. Black box ensures the system meets user needs; white box ensures the code is robust and free of internal defects.

11. Explain ISO 9001 quality standards.

[10]

Answer: ISO 9001 Quality Standards:
ISO 9001 is an international standard that specifies requirements for a Quality Management System (QMS). It is part of the ISO 9000 family of standards and is the only standard in the family that can be certified. Organizations use ISO 9001 to demonstrate their ability to consistently provide products and services that meet customer and regulatory requirements.

Key Principles of ISO 9001:
1. Customer Focus: Understand current and future customer needs, meet customer requirements, and strive to exceed expectations.
2. Leadership: Establish unity of purpose and direction. Leaders create an environment where people are engaged in achieving quality objectives.
3. Engagement of People: Competent, empowered, and engaged people at all levels are essential to enhance the organization's capability to create value.
4. Process Approach: Consistent and predictable results are achieved more effectively when activities are understood and managed as interrelated processes.
5. Improvement: Ongoing improvement of the organization's overall performance should be a permanent objective.
6. Evidence-Based Decision Making: Decisions based on analysis and evaluation of data are more likely to produce desired results.
7. Relationship Management: An organization and its external providers (suppliers, partners) are interdependent, and a mutually beneficial relationship enhances the ability of both to create value.

ISO 9001:2015 Structure (High-Level Structure):
• **Clause 4:** Context of the Organization
• **Clause 5:** Leadership
• **Clause 6:** Planning (including risk-based thinking)
• **Clause 7:** Support (resources, competence, awareness, communication)
• **Clause 8:** Operation (process execution)
• **Clause 9:** Performance Evaluation (monitoring, measurement, analysis, internal audit)
• **Clause 10:** Improvement (nonconformity, corrective action, continual improvement)

Benefits of ISO 9001 Certification:
• Improved product and service quality.
• Increased customer satisfaction and trust.
• Better process efficiency and waste reduction.
• Enhanced market credibility and competitive advantage.
• Foundation for other standards (ISO 27001, ISO 14001).

ISO 9001 in Software Engineering:
Software companies adopt ISO 9001 to formalize their development processes, ensure traceability of requirements, manage changes, and maintain documentation. It complements agile practices by adding structure to quality assurance and continuous improvement.